

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5115

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Scenario-Based: 20%

QUESTIONS: 2

Total Marks: 20

Annexure A

Code of Conduct:

I M Nishanth (192511154) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A

Scenario-based

1. Secure Data Transmission using Block Cipher

A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

- A. Explain why this issue occurs.
- B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System

A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

- A. Identify the most suitable algorithm for this scenario.
- B. Justify your choice based on security strength and efficiency.

SIMATS ENGINEERING

ASSESSMENT TOOL 2 – CSA5115 CRYPTOGRAPHY AND NETWORK SECURITY (CO2)

Annexure A – Questions 1 & 2: Worked Solutions

Question 1: Secure Data Transmission using Block Cipher

Scenario: A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A) Why This Issue Occurs

Answer: ECB encrypts each block independently, so identical plaintext blocks always produce identical ciphertext blocks – exposing the data's structure.

Electronic Codebook (ECB) mode encrypts every plaintext block separately, using the same key and no chaining or randomization between blocks: $C_i = E_k(P_i)$. Because the transformation applied to each block is identical and independent of the other blocks, any two plaintext blocks that are identical will always produce identical ciphertext blocks.

Financial data frequently contains repeated or structured values – for example, repeated transaction amounts, standard header fields, zero-padding, or recurring account/reference codes. When such data is encrypted under ECB, this repetition and structure carry straight through into the ciphertext, even though the actual bit values are technically encrypted. An attacker observing the ciphertext can therefore infer patterns, detect repeated records, and deduce structural information about the underlying financial data – without ever having to break the encryption algorithm itself. This is the well-known weakness commonly illustrated by the “ECB penguin” effect, where a recognizable image encrypted in ECB mode remains visibly recognizable in its ciphertext form.

B) A More Secure Block Cipher Mode, and Justification

Answer: Use Cipher Block Chaining (CBC) mode – or, where authentication is also needed, Galois/Counter Mode (GCM).

CBC mode chains every block's encryption to the one before it: each plaintext block is XORed with the previous ciphertext block before being encrypted, $C_i = E_k(P_i \oplus C_{i-1})$, with a random/unique Initialization Vector (IV) used for the very first block. Because of this dependency, even two identical plaintext blocks will produce completely different ciphertext

blocks as long as their position or preceding context differs – directly eliminating the pattern-leakage problem observed under ECB.

For this specific scenario – repeated financial data blocks transmitted over a network – CBC is the appropriate direct fix because it requires no change to the underlying block cipher itself, only the mode of operation, and it is a well-established, widely supported standard. As a further enhancement, since financial transactions also require guarantees that data has not been tampered with in transit, an Authenticated Encryption mode such as GCM (Galois/Counter Mode) could be recommended instead: GCM provides the same pattern-hiding diffusion as CBC while also generating an authentication tag that lets the receiver verify both confidentiality and integrity of the data in a single pass, and it supports parallel processing for better throughput than CBC – making it an even stronger choice for high-value financial transmissions.

Question 2: Choosing an Encryption Algorithm for Banking System

Scenario: A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A) Most Suitable Algorithm

Answer: Blowfish is the most suitable algorithm for this scenario.

Algorithm	Key Size	Security Level	Relative Speed
DES	56 bits	Weak – practically brute-forceable today	Fast
3-DES	Up to 168 bits (effective ≈ 112 bits)	Strong, but with high computational overhead	Slow ($\approx 3 \times$ DES, triple-pass structure)
Blowfish	Variable, 32–448 bits (commonly 128 bits)	Strong – no known practical cryptanalytic break	Fast – efficient Feistel structure, often faster than DES in software

B) Justification: Security Strength and Efficiency

1. DES is ruled out despite being fast, because its 56-bit key length is now considered cryptographically weak and can be brute-forced with modern computing resources – unacceptable for protecting banking transactions.
2. 3-DES is ruled out as the primary choice because, although it offers strong effective security, applying DES three times in sequence makes it roughly three times slower than DES, which fails to meet the “moderate performance” requirement, especially for a banking system that must process a high volume of transactions with low latency.
3. Blowfish satisfies both requirements simultaneously: its variable key length (up to 448 bits) allows the bank to configure strong security well beyond DES/3-DES equivalence, it has withstood decades of cryptanalysis without a practical break, and its simple, efficient Feistel-network design makes it significantly faster than 3-DES – and often faster than DES itself in software implementations.

Given that the banking application explicitly requires both high security and moderate (not minimal) performance, Blowfish offers the best overall balance of the three options, making it the recommended algorithm for encrypting customer transactions in this scenario.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

